

Capability | Description

User authentication | Users can register, log in, and access protected document and QA endpoints through bearer tokens.

File upload | Users can upload TXT, PDF, PNG, JPG, and JPEG files. Unsupported formats are rejected.

Document management | Users can view document names, types, sizes, processing status, errors, and update times.

Text extraction | TXT files are decoded directly, PDFs are parsed with PyMuPDF, and images are OCR-processed with Tesseract where installed.

Text cleaning and chunking | Extracted text is normalised and split into smaller chunks with document and page metadata.

Retrieval | The system retrieves relevant chunks using optional semantic retrieval and a local lexical fallback.

Question answering | The backend constructs a prompt using retrieved chunks and calls DeepSeek when configured.

Citation display | Answers include citations containing document name, page number, chunk identifier, preview text, and score.

Graceful fallback | If the LLM key is missing or the external call fails, the system returns a local evidence-based answer rather than failing silently.

Layer | Main responsibility | Main technologies
Frontend layer | Login, upload, document management, chat, citations | React, TypeScript, Vite
API layer | HTTP endpoints, authentication, request validation, response formatting | FastAPI, Pydantic
Persistence layer | User, document, and chunk metadata | SQLite, SQLAlchemy
File storage layer | Original uploaded files and retrieval manifests | Local file system
Processing layer | Text extraction, OCR, cleaning, and chunking | PyMuPDF, pytesseract,

Tesseract

Retrieval layer | Chunk search and ranking | SentenceTransformers, FAISS, lexical fallback
Generation layer | Prompt construction and answer generation | DeepSeek API, fallback answer logic

Router | Endpoints | Responsibility

Authentication router | Register, login, current user | Account creation, password verification, token issuing
Documents router | Upload, list, get, delete, reprocess | File management and processing orchestration
QA router | Query endpoint | Question validation and answer response
Health endpoint | Health/configuration status | Basic operational check

Service | Responsibility

Storage service | File type validation, filename sanitisation, upload saving, file deletion
Document processor | TXT decoding, PDF parsing, image OCR, text cleaning
Chunking service | Chunk generation with document, page, and index metadata
Vector service | Chunk persistence, retrieval, semantic search attempt, lexical fallback
LLM service | DeepSeek request construction and response extraction
RAG service | Retrieval orchestration, citation formatting, fallback answer generation

Entity | Purpose

User | Stores user identity and hashed password
Document | Stores uploaded document metadata, owner, status, error message, and storage filename
DocumentChunk | Stores extracted chunks with owner, document, page, and text

Component or page	Responsibility
`App`	Authentication state, top-level navigation, session persistence
`LoginPage`	Register/login form and authentication feedback
`DocumentsPage`	Document list, search, refresh, delete and reprocess actions
`UploadPanel`	File selection, client-side format validation, upload status
`DocumentTable`	Document metadata, status badges, row actions
`ChatPage`	Chat state and QA request handling
`ChatWindow`	Message display, question input, loading state
`CitationList`	Expandable citation previews
`api.ts`	Typed HTTP client for backend endpoints

Method and path | Purpose

`GET /health` | Check backend status and DeepSeek configuration state
`POST /auth/register` | Register a user
`POST /auth/login` | Log in and receive bearer token
`GET /auth/me` | Retrieve current user
`POST /documents/upload` | Upload and process a document
`GET /documents` | List current user's documents
`GET /documents/{document\_id}` | Retrieve document metadata
`DELETE /documents/{document\_id}` | Delete document and chunks
`POST /documents/{document\_id}/reprocess` | Re-run processing
`POST /qa/query` | Ask a question and receive answer plus citations

Area | Technology
Frontend | React, TypeScript, Vite
Backend | FastAPI, Python
API validation | Pydantic models through FastAPI
Database | SQLite through SQLAlchemy
File storage | Local file system
PDF processing | PyMuPDF
Image OCR | Tesseract through pytesseract
Retrieval | SentenceTransformers and FAISS when available, lexical fallback otherwise
LLM integration | DeepSeek official chat completions API
Testing | pytest, FastAPI TestClient, TypeScript build
Environment management | uv for backend, npm for frontend

Path | Purpose
`apps/api` | FastAPI backend
`apps/web` | React frontend
`data/uploads` | Local uploaded files, ignored by Git
`data/extracted` | Reserved for extracted text cache, ignored by Git
`data/indexes` | Retrieval manifests and future index files, ignored by Git
`docs` | Architecture, API contract, and report documents
`tools` | Utility scripts such as the DeepSeek console tester
`.env.example` | Template configuration

``.gitignore`` | Prevents secrets, generated files, and local data from being committed

Variable	Purpose
<code>`SECRET_KEY`</code>	Token signing secret for local authentication
<code>`DEEPSEEK_API_KEY`</code>	DeepSeek official API key
<code>`DEEPSEEK_MODEL`</code>	Model name, currently <code>`deepseek-v4-pro`</code>
<code>`EMBEDDING_MODEL`</code>	Optional SentenceTransformers model
<code>`CHUNK_SIZE`</code>	Approximate chunk length
<code>`CHUNK_OVERLAP`</code>	Overlap between chunks
<code>`CORS_ORIGINS`</code>	Allowed frontend origins

Milestone | Outcome

Requirements and proposal | Defined target users, local-first RAG scope, technologies, and exclusions

Architecture design | Established layered architecture and local data model

Backend foundation | Implemented authentication, configuration, database, and router structure

Document processing | Added file upload, PDF parsing, text extraction, OCR path, and status handling

Retrieval and RAG | Added chunking, retrieval, DeepSeek generation, and fallback answers

Frontend workflow | Added login, upload, document list, chat, and citation UI

Verification | Added automated tests and build checks

Documentation | Created API contract, architecture notes, README, utility script, and final report

Requirement | Outcome

Upload PDFs, images, and text files | Implemented for TXT, PDF, PNG, JPG, and JPEG

Local storage | Implemented through local data directories and SQLite

Document parsing | Implemented for TXT and PDF; image OCR supported when Tesseract is installed

Text cleaning | Implemented with whitespace, null-character, and line normalisation

Chunking | Implemented with configurable chunk size and overlap

Vector retrieval | Designed with optional SentenceTransformers and FAISS, with lexical fallback

Natural language QA | Implemented through QA endpoint and chat UI
Source traceability | Implemented through structured citations
Basic web interface | Implemented with login, document management, and chat views
Error handling | Implemented for common upload, processing, OCR, and LLM failures
Authentication | Implemented with registration, login, bearer token, and ownership checks
Local-first privacy | Implemented for files, metadata, retrieval, and optional external generation

Test area | Evidence

Authentication | Register, login, token use, and current-user endpoint
Text processing | TXT processing and cleaned text output
Chunking | Chunk creation with document and page metadata
Document upload | Uploading a text file and receiving indexed status
QA fallback | Asking a question and receiving answer plus citations

Check | Result

Backend pytest suite | 3 tests passed
Backend static check | Passed
Frontend TypeScript and production build | Passed

Assessment expectation | Where addressed
Introduction and system overview | Section 1
System architecture and design decisions | Section 2
Implementation and technology stack | Section 3
Development process and reflection | Section 4
Evaluation of outcomes | Section 5
Conclusion and future work | Section 6
References below 20 items | References section
No source code included | The report describes behaviour, design, and interfaces without code listings
AI-assisted tool acknowledgement | Section 4.6

Feature | Implemented state
User registration and login | Implemented
Protected document and QA endpoints | Implemented
TXT upload and extraction | Implemented
PDF text extraction | Implemented through PyMuPDF
Image OCR | Implemented when Tesseract is installed
Text cleaning | Implemented
Chunking with page metadata | Implemented
Chunk persistence | Implemented through SQLite
Local retrieval | Implemented
Optional semantic retrieval | Designed through SentenceTransformers and FAISS path
DeepSeek answer generation | Implemented when `DEEPSEEK\_API\_KEY` is configured
Fallback answer | Implemented
Citation display | Implemented
Frontend document management | Implemented
Frontend chat interface | Implemented
Automated backend tests | Implemented
Frontend production build check | Implemented